

LECTURE 9

Transformers & Attention

Embeddings, positional encoding, queries, keys and values, multi-head attention — and the architecture of *Attention Is All You Need*

AK DHAMIJA, Researcher & Instructor

AI & Machine Learning

dhamija.ak@gmail.com • 9811453163

Learning Objectives & Session Structure

By the end of this lecture you will be able to

1. Explain how a transformer reads every word of a sentence at the same time
2. Use word vectors: why *tiger* $-$ *man* $+$ *woman* $\approx$ *tigress*
3. Compute sinusoidal positional encodings and explain why they work
4. Compute scaled dot-product attention with Q, K, V by hand
5. Run a two-head multi-head attention example and its Add & Norm
6. Explain every block of Figure 1 of Vaswani et al. (2017)
7. Distinguish BERT, GPT and encoder–decoder models; assess defence uses

Session Structure (45 min)

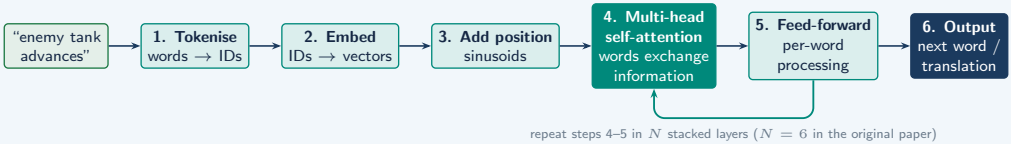
- **5 min** — Opening Q&A: the intuitions of L8
- **15 min** — **Part 1:** parallel reading; embeddings and word arithmetic; positional encoding; Q/K/V worked example
- **15 min** — **Part 2:** multi-head worked example; the Vaswani architecture block by block; GPT vs BERT; defence uses
- **10 min** — Auto-quiz (15 questions)

This is the densest deck in the course. Where time allows, run it as two sessions (Part 1, then Part 2), with the quiz at the end of the second.

Running example throughout: the sentence "*enemy tank advances*" — 3 tokens, 4-dimensional toy vectors, every number checked.

Pre-requisites: Lecture 6 (hidden layers, backpropagation) and Lecture 8 (the intuitions).

The Big Picture in One Slide



Parallel

All words enter together as one matrix — no word-by-word loop.

Embedding

Each token becomes a vector; directions in the space carry meaning.

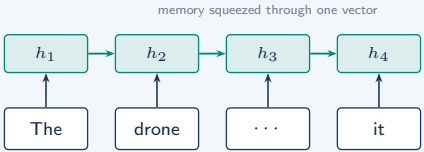
Position

Sine and cosine waves stamp each vector with its place in the sentence.

Attention

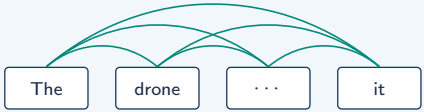
Each word decides which other words matter to it and blends their information.

RNN: read one word at a time



- **Sequential**: word 50 waits for words 1–49 — no parallel training on GPUs
- **Forgetting**: information from early words fades through many steps (vanishing gradients)
- **Bottleneck**: the whole past is compressed into one hidden vector h_t

Transformer: every word sees every word

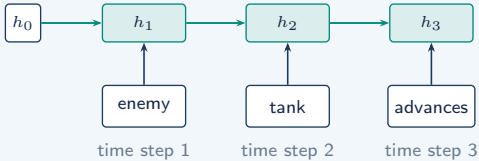


- **Parallel**: all positions computed at once — fast on GPUs
- **Direct links**: "it" connects to "drone" in *one* step, however far apart
- **Learned relevance**: the model learns which links matter

Vaswani et al., *Attention Is All You Need* (NeurIPS 2017): drop recurrence entirely and use attention alone.

Running Example: Reading Every Word at the Same Time

RNN: three steps, each waiting for the last



$h_t = \tanh(W_h h_{t-1} + W_x x_t)$: h_2 cannot be computed until h_1 exists, so a 1 000-word report needs 1 000 sequential steps per layer.

Path from "enemy" to "advances": 2 hops through h_1 and h_2. For words 500 apart: 500 hops.

Transformer: one matrix, one operation

Stack the three token vectors as the rows of one matrix X and multiply once:

$$\underbrace{\begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 \end{pmatrix}}_{X\ (3\times 4)} \begin{matrix} \leftarrow \text{enemy} \\ \leftarrow \text{tank} \\ \leftarrow \text{advances} \end{matrix} \underbrace{\begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 0 \end{pmatrix}}_{W^Q\ (4\times 2)} = \underbrace{\begin{pmatrix} 2 & 0 \\ 0 & 1 \\ 1 & 0 \end{pmatrix}}_{Q\ (3\times 2)}$$

All three queries appear in *one* step. Likewise $S = QK^\top$ scores all $3 \times 3 = 9$ word pairs in one matrix product.

On a GPU the rows are computed simultaneously, so a 1 000-token passage costs **one** parallel step per layer, and every word is **one** hop from every other.

During training even the decoder handles all target positions at once; the causal mask (later) stops each position seeing its future.

The Paper's Own Comparison (Vaswani et al., Table 1)

Layer type	Complexity per layer	Sequential operations	Maximum path length
Self-attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k n)$

n = sequence length, d = representation size, k = convolution kernel width

Sequential operations

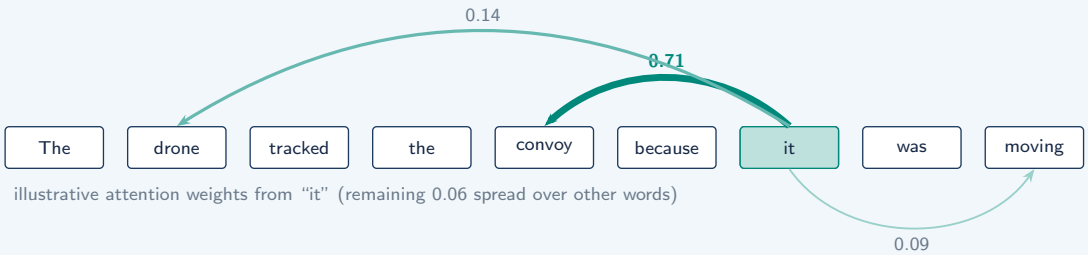
Steps that must wait for each other. $O(1)$ means the whole sentence is processed in parallel — the training-speed breakthrough.

Maximum path length

Hops a signal needs between the two most distant words. Short paths make long-range dependencies easy to learn.

The price: n^2 — every word scores every word. Typical sentences ($n \approx 30$) are far shorter than $d = 512$, so it is cheap; very long documents are expensive.

The Intuition: An Intelligence Analyst Reading a Report



illustrative attention weights from "it" (remaining 0.06 spread over other words)

What attention does

To understand "it", the model asks *which words could "it" refer to?* and pulls most of its meaning from "convoy" — "moving" fits a convoy being tracked.

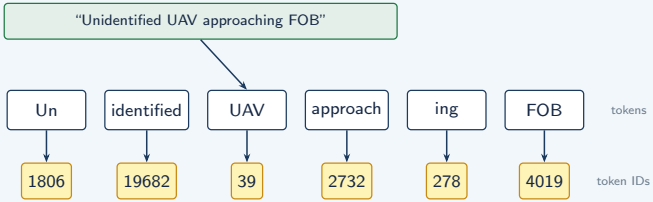
The analyst analogy

An analyst reading "*it* was moving" does not reread the report in order: they glance back to the few words that resolve the reference.

Change one word — "because *it* was **airborne**" — and the weight should move to "drone". Context, not a fixed rule, decides.

Attention is a **learned, soft cross-referencing** step: every word receives a weighted summary of the words most relevant to it, with weights that sum to 1.

Step 1 — Tokenisation: Text to Numbers



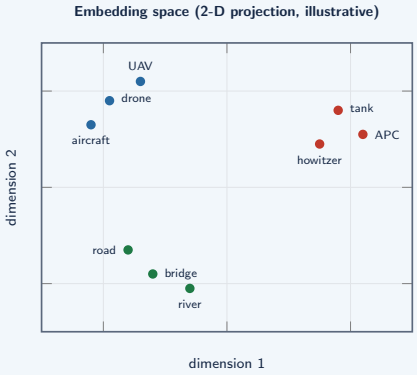
Sub-word tokens

Models use a fixed vocabulary of pieces (the original paper used byte-pair encoding with about 37 000 tokens shared by English and German). Common words are one token; rare words split into pieces — "Unidentified" → "Un" + "identified".

Note: the IDs shown are illustrative — every tokenizer assigns its own. The model never sees letters, only these integers. Rule of thumb for English: 1 token ≈ 3/4 of a word.

Military jargon, call-signs and grid references are often split into many tokens — a hidden cost in context length and accuracy.

Step 2 — Embeddings: Placing Words in a Meaning Space



An embedding is a learned lookup table

Matrix $E \in \mathbb{R}^{V \times d}$: row i is the vector for token i. "Embedding" = picking a row.

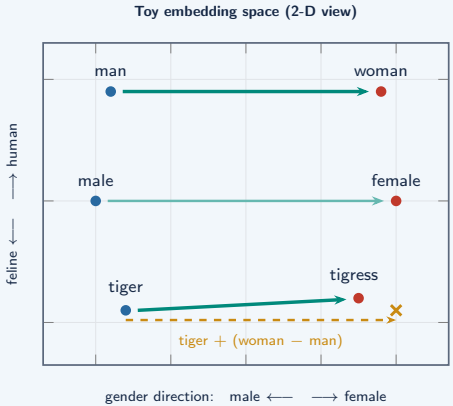
$$x_{\text{tank}} = E[\text{ID}_{\text{tank}}] = (0.21,\ -1.30,\ 0.75,\ \dots) \in \mathbb{R}^d$$

$d_{\text{model}} = 512$ in the original transformer, 768 in BERT-base, $12\,288$ in GPT-3.

Similar meaning ⇒ nearby vectors. Nobody sets these positions: they emerge from training, because words used in similar contexts must produce similar predictions.

Similarity is measured by the dot product or cosine $\cos\theta = \dfrac{x \cdot y}{\|x\|\|y\|}$ — the same operation attention uses.

Directions Carry Meaning: man → woman ≈ tiger → tigress



The same arrow, three times

The step from *man* to *woman*, from *male* to *female* and from *tiger* to *tigress* points the **same way**: training has made "male → female" a *direction* in the space.

VECTOR ARITHMETIC

$$\overrightarrow{\text{tiger}} - \overrightarrow{\text{man}} + \overrightarrow{\text{woman}} \approx \overrightarrow{\text{tigress}}$$
$$\overrightarrow{\text{man}} - \overrightarrow{\text{male}} + \overrightarrow{\text{female}} \approx \overrightarrow{\text{woman}}$$
$$\overrightarrow{\text{king}} - \overrightarrow{\text{man}} + \overrightarrow{\text{woman}} \approx \overrightarrow{\text{queen}}$$

Read the first line as: "start at *tiger*, remove what makes *man* male, add what makes *woman* female" — the dashed orange arrow lands next to *tigress*.

Worked Analogy: Computing tiger − man + woman

Toy 4-D vectors (gender, human, feline, royalty)

word	gender	human	feline	royal
man	0.90	0.95	0.05	0.05
woman	−0.90	0.95	0.05	0.05
tiger	0.80	0.05	0.95	0.00
tigress	−0.75	0.10	0.90	0.05
lioness	−0.80	0.05	0.90	0.30
king	0.85	0.90	0.00	0.95
queen	−0.85	0.90	0.00	0.95
female	−1.00	0.10	0.10	0.00

Gender direction: $\overrightarrow{\text{woman}} - \overrightarrow{\text{man}} = (-1.80,\ 0,\ 0,\ 0)$

Step 1: the arithmetic

$(0.80, 0.05, 0.95, 0) + (-1.80, 0, 0, 0) =$
$(\text{-}\mathbf{1.00},\ \mathbf{0.05},\ \mathbf{0.95},\ \mathbf{0.00})$

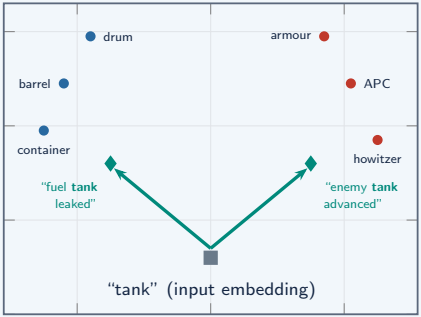
Step 2: nearest word by cosine similarity

candidate	$\cos(\text{result}, \cdot)$
tigress	**0.991**
lioness	0.967
female	0.789

Same toy space: *king* − *man* + *woman* → **queen** (0.999); *man* − *male* + *female* → **woman** (0.995).

Reality check: real embeddings have hundreds of dimensions with no human-readable labels (ours are labelled only for teaching). Analogies hold *approximately*, the input words are excluded from the search, and the same geometry absorbs biases in the training text (Mikolov et al. 2013; Bolukbasi et al. 2016).

Contextual vectors for "tank" (illustrative)



Two kinds of space

- **Word space**: the input embedding table gives *one* fixed vector per token — "tank" is the same in every sentence
- **Context space**: after attention layers, each token's vector has absorbed its neighbours — the *contextual embedding*

Attention *moves* the ambiguous vector: with "fuel" and "leaked" it drifts towards containers; with "enemy" and "advanced" towards armour. Directions like male $\rightarrow$ female still exist, but positions now depend on the sentence.

This is why an intelligence summariser can tell an armoured "tank" from a fuel "tank" — or "fire" (weapon) from "fire" (blaze).

Attention is order-blind. It treats its input as a *set*: scores depend only on which vectors are present, not where. "**Blue** engages **Red**" and "**Red** engages **Blue**" would produce identical word vectors — just listed in a different order.

The fix in the paper

Before the first layer, add a *position vector* p_t of the same size to each token embedding:

$$\text{input}_t = \sqrt{d_{\text{model}}}\, x_t + p_t$$

($\sqrt{d_{\text{model}}}$ scaling as in the paper). Nothing is concatenated: position and meaning share the same d_{model} numbers.

What a good position code needs

1. A **unique** code for every position
2. **Bounded** values, so it does not swamp the word meaning
3. The **same distance** between positions 3 and 5 as between 103 and 105
4. Works for sentences **longer** than any seen in training

Simply using $p_t = t$ fails rule 2 (position 1 000 would dwarf the meaning); using t/n fails rule 3 (depends on sentence length). Vaswani et al. chose **sine and cosine waves**.

The Sinusoidal Positional Encoding

VASWANI ET AL. (2017), SECTION 3.5

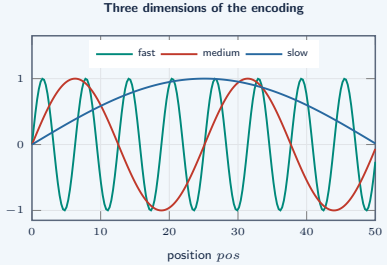
$$PE_{(pos,\,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

$$PE_{(pos,\,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

- pos = position in the sentence (0, 1, 2, …)
- i = index of a dimension *pair*; even dimension $2i$ gets sine, odd dimension $2i+1$ gets cosine of the same angle
- frequency $\omega_i = 1/10000^{2i/d_{\text{model}}}$ falls from 1 to $1/10000$; wavelengths grow geometrically from 2π to $10000 \cdot 2\pi$

pair i	ω_i	**wavelength (tokens)**
0	1	6.3
64	0.1	63
128	0.01	628
255	≈ 0.0001	$\approx 60\,600$

($d_{\text{model}} = 512$)



Clock analogy: the fast dimensions are a second hand (they tell neighbours apart); the slow ones are an hour hand (they tell the start of a long report from its end). Reading all hands together gives a unique time — a unique position.

With $d_{\text{model}} = 4$ there are two frequency pairs

pair 0: $\omega_0 = 1/10000^{0/4} = 1$; pair 1: $\omega_1 = 1/10000^{2/4} = 0.01$

$p_{pos} = (\sin pos,\ \cos pos,\ \sin 0.01pos,\ \cos 0.01pos)$ (radians)

	pos	$\sin pos$	$\cos pos$	$\sin .01pos$	$\cos .01pos$
enemy	0	0.0000	1.0000	0.0000	1.0000
tank	1	0.8415	0.5403	0.0100	1.0000
advances	2	0.9093	-0.4161	0.0200	0.9998

Fast pair: $0 \to 0.84 \to 0.91$ (big changes between neighbours). Slow pair: $0 \to 0.01 \to 0.02$ (barely moves — it separates positions hundreds of tokens apart).

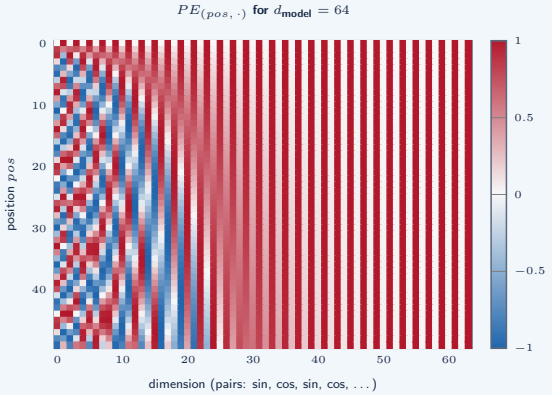
Add to the word embeddings E

$$\underbrace{\begin{pmatrix}1 & 0 & 1 & 0\\ 0 & 1 & 0 & 1\\ 1 & 0 & 0 & 1\end{pmatrix}}_{E} + \underbrace{\begin{pmatrix}0 & 1 & 0 & 1\\ .84 & .54 & .01 & 1\\ .91 & -.42 & .02 & 1\end{pmatrix}}_{P}$$

$$= \underbrace{\begin{pmatrix}1.00 & 1.00 & 1.00 & 1.00\\ 0.84 & 1.54 & 0.01 & 2.00\\ 1.91 & -0.42 & 0.02 & 2.00\end{pmatrix}}_{X=E+P} \begin{matrix}\text{enemy}\\ \text{tank}\\ \text{advances}\end{matrix}$$

Each row now carries *what* the word is and *where* it is. (Here $\sqrt{d_{\text{model}}} = 2$ scaling of E is omitted to keep the numbers simple.)

The Full Encoding as a Picture: 50 Positions × 64 Dimensions



How to read it

Each **row** is the code added to one word. Left columns flip colour rapidly from row to row (fast waves); right columns change slowly or hardly at all (slow waves).

Every row is different — a unique "barcode" per position — and every value stays in $[-1, 1]$, so it never swamps the meaning.

The paper also tried *learned* position vectors and got nearly identical results (Table 3, row E); it kept sinusoids because they may extrapolate to sequences longer than those seen in training.

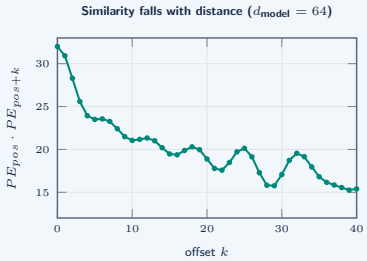
Shift by k positions = a fixed rotation

For one frequency pair, using $\sin(a+b)$ and $\cos(a+b)$ identities:

$$\begin{pmatrix} \sin \omega(pos+k) \\ \cos \omega(pos+k) \end{pmatrix} = \underbrace{\begin{pmatrix} \cos \omega k & \sin \omega k \\ -\sin \omega k & \cos \omega k \end{pmatrix}}_{\text{depends only on } k} \begin{pmatrix} \sin \omega\, pos \\ \cos \omega\, pos \end{pmatrix}$$

So PE_{pos+k} is a **linear function** of PE_{pos} — the paper's stated reason: the model "could easily learn to attend by relative positions".

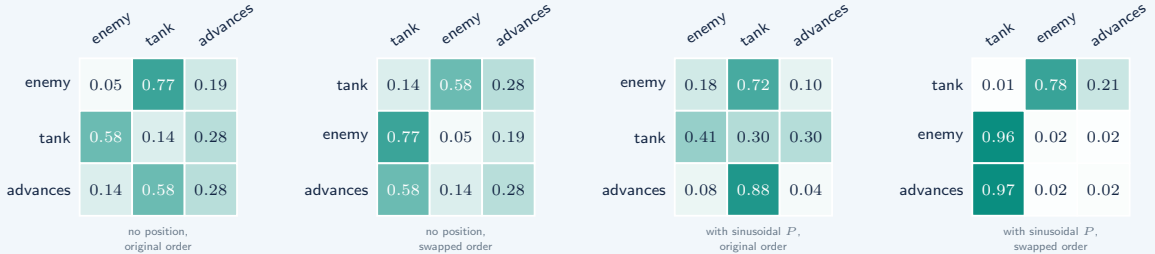
Check ($\omega = 1$, $pos = 3$, $k = 2$): rotating $(\sin 3, \cos 3)$ by angle 2 gives $(-0.9589,\ 0.2837) = (\sin 5, \cos 5)$ ✓



$PE_{pos} \cdot PE_{pos+k} = \sum_i \cos(\omega_i k)$ depends **only on the offset** k: positions 5 & 10 score exactly like 30 & 35 (23.50). Nearby words look more alike than distant ones.

Running Example: Positions Change What Attention Sees

Same attention weights W^Q, W^K, W^V as the worked example that follows; compare the original order with the swapped sentence "tank enemy advances".



Without position the swapped sentence gives *exactly the same numbers*, merely re-ordered (enemy→tank is 0.77 and tank→enemy 0.58 either way). The model cannot tell the two word orders apart.

With position the weights change when the order changes: enemy→tank is 0.72 in the original order but 0.96 when swapped; tank→enemy rises from 0.41 to 0.78. Word order now influences meaning.

Query, Key, Value: The Intelligence-Archive Analogy



Query q

What am I looking for? Produced by the word that is *asking*.

Key k

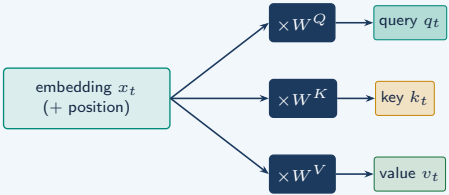
What do I contain? The label each word offers for matching. Match score $= q \cdot k$.

Value v

What do I hand over? The content passed on, weighted by the match.

A database returns *one* exact match. Attention is a **soft** lookup: it returns a *weighted blend* of all values, and because it is differentiable it can be trained by backpropagation.

Where Do Q, K and V Come From?



three learned weight matrices

FOR ALL TOKENS AT ONCE

$$Q = XW^Q, \qquad K = XW^K, \qquad V = XW^V$$

Same word, three roles

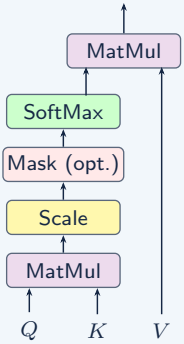
One embedding is projected three different ways. "tank" as a *query* might look for "who owns me?"; as a *key* it advertises "I am an armoured vehicle"; as a *value* it carries its meaning to whoever attends to it.

Shapes

X: $n \times d_{\text{model}}$ (n tokens)
W^Q, W^K: $d_{\text{model}} \times d_k$; W^V: $d_{\text{model}} \times d_v$
Q, K: $n \times d_k$; V: $n \times d_v$ (paper: $d_k = d_v = 64$)

W^Q, W^K, W^V are ordinary weights, learned by backpropagation exactly like the weights in Lectures 6 and 7.

Scaled Dot-Product Attention (Vaswani et al., Figure 2, left)



redrawn after Vaswani et al. (2017)

SCALED DOT-PRODUCT ATTENTION

$$\text{Attention}(Q, K, V) \quad = \quad \text{softmax}\left(\frac{QK^{\mathsf{T}}}{\sqrt{d_k}}\right) V$$

Reading the diagram bottom to top

1. **MatMul** QK^{T}: score every query against every key ($n \times n$)
2. **Scale** by $1/\sqrt{d_k}$: keep the scores in a moderate range
3. **Mask (opt.)**: in the decoder, set scores for future words to $-\infty$
4. **SoftMax** along each row: turn scores into weights that sum to 1
5. **MatMul** with V: blend the value vectors with those weights

softmax$(s)_j = e^{s_j} / \sum_m e^{s_m}$ — positive, sums to 1, exaggerates the largest scores. **Self-attention:** Q, K and V all come from the same sentence.

Worked Example: Set-Up ("enemy tank advances")

Input vectors X (3 tokens, $d_{\text{model}} = 4$)

$$X = \begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 \end{pmatrix} \begin{matrix} \leftarrow \text{enemy} \\ \leftarrow \text{tank} \\ \leftarrow \text{advances} \end{matrix}$$

Integer toy vectors keep every step hand-computable; think of them as embedding + position already combined. The previous slide showed the same weights applied to the true $E + P$.

Learned projections ($d_k = d_v = 2$)

$$W^Q = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 0 \end{pmatrix} \quad W^K = \begin{pmatrix} 0 & 1 \\ 1 & 0 \\ 0 & 1 \\ 1 & 0 \end{pmatrix} \quad W^V = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \\ 0 & 1 \end{pmatrix}$$

Plan

- Compute Q, K, V (3 matrix products)
- Scores QK^{T}, scaled by $\sqrt{2} = 1.4142$
- Softmax each row $\rightarrow$ attention weights A
- Output $Z = AV$: a new, context-aware vector per word

Each row of X times the weight matrix. For "enemy" $= (1, 0, 1, 0)$: rows 1 and 3 of W^Q add up, $q_{\text{enemy}} = (1, 0) + (1, 0) = (2, 0)$.

$$Q = XW^Q = \begin{pmatrix} 2 & 0 \\ 0 & 1 \\ 1 & 0 \end{pmatrix} \qquad K = XW^K = \begin{pmatrix} 0 & 2 \\ 2 & 0 \\ 1 & 1 \end{pmatrix} \qquad V = XW^V = \begin{pmatrix} 2 & 1 \\ 0 & 2 \\ 1 & 1 \end{pmatrix}$$

Check: tank

$x = (0, 1, 0, 1)$ selects rows 2 and 4.
$q = (0, 1) + (0, 0) = (0, 1)$
$k = (1, 0) + (1, 0) = (2, 0)$
$v = (0, 1) + (0, 1) = (0, 2)$

Check: advances

$x = (1, 0, 0, 1)$ selects rows 1 and 4.
$q = (1, 0) + (0, 0) = (1, 0)$
$k = (0, 1) + (1, 0) = (1, 1)$
$v = (1, 0) + (0, 1) = (1, 1)$

Notice $q_{\text{enemy}} = (2, 0)$ points the same way as $k_{\text{tank}} = (2, 0)$: "enemy" is about to find "tank" highly relevant.

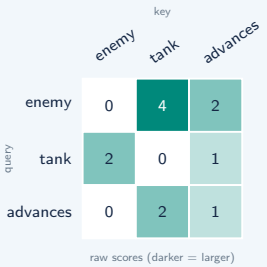
Raw scores $S = QK^{\mathsf{T}}$ (row = query, column = key)

$$S = \begin{pmatrix} 2 & 0 \\ 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 0 & 2 & 1 \\ 2 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 0 & 4 & 2 \\ 2 & 0 & 1 \\ 0 & 2 & 1 \end{pmatrix}$$

e.g. enemy→tank: $q_{\text{enemy}} \cdot k_{\text{tank}} = (2)(2) + (0)(0) = 4$

Scaled scores $S/\sqrt{d_k} = S/1.4142$

$$\begin{pmatrix} 0 & 2.8284 & 1.4142 \\ 1.4142 & 0 & 0.7071 \\ 0 & 1.4142 & 0.7071 \end{pmatrix}$$



Scores are dot products: large when the query and key vectors point the same way.

Row "enemy": exponentiate, then normalise

$$(e^0,\ e^{2.8284},\ e^{1.4142}) = (1,\ 16.9188,\ 4.1133)$$

$$\text{sum} = 22.0321$$

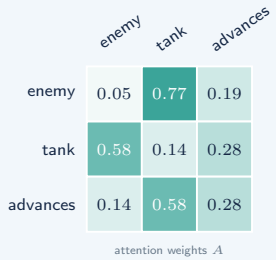
$$a_{\text{enemy}} = \left(\tfrac{1}{22.03},\ \tfrac{16.92}{22.03},\ \tfrac{4.11}{22.03}\right)$$

$$= (\mathbf{0.0454},\ \mathbf{0.7679},\ \mathbf{0.1867})$$

All three rows

$$A = \begin{pmatrix} 0.0454 & 0.7679 & 0.1867 \\ 0.5760 & 0.1400 & 0.2840 \\ 0.1400 & 0.5760 & 0.2840 \end{pmatrix}$$

Every row sums to 1: each word distributes 100% of its attention.



attention weights A

Reading the map: "enemy" looks at "tank" (77%), "tank" looks back at "enemy" (58%), and the verb "advances" looks to its subject "tank" (58%).

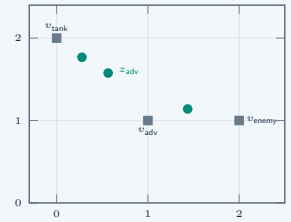
Row "advances": weighted sum of the value vectors

$$\begin{aligned} z_{\text{adv}} &= 0.1400\, v_{\text{enemy}} + 0.5760\, v_{\text{tank}} + 0.2840\, v_{\text{adv}} \\ &= 0.14(2,1) + 0.576(0,2) + 0.284(1,1) \\ &= (0.28 + 0 + 0.284,\quad 0.14 + 1.152 + 0.284) \\ &= (\mathbf{0.5641},\ \mathbf{1.5760}) \end{aligned}$$

All three outputs

$$Z = AV = \begin{pmatrix} 0.2775 & 1.7679 \\ 1.4359 & 1.1400 \\ 0.5641 & 1.5760 \end{pmatrix} \begin{matrix} \leftarrow \text{enemy} \\ \leftarrow \text{tank} \\ \leftarrow \text{advances} \end{matrix}$$

Values v (squares) and outputs z (dots)



Each output is pulled toward the values of the words it attended to: z_{advances} now carries mostly "tank" information. The verb "knows" what is advancing.

Dot products grow with dimension

If query and key entries are independent with mean 0 and variance 1, then $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$ has variance d_k, so its typical size is $\sqrt{d_k}$ (the paper's footnote 4).

d_k	std of $q \cdot k$ (simulated)	$\sqrt{d_k}$
2	1.42	1.41
64	8.04	8.00
512	22.59	22.63

Dividing by $\sqrt{d_k}$ brings scores back to unit scale, whatever the model size.

Without scaling large scores push softmax into saturation: one weight ≈ 1, the rest ≈ 0. The softmax gradient then vanishes and learning stalls.

Our example, row "enemy"

- Unscaled softmax of $(0, 4, 2)$: $(0.016,\ \mathbf{0.867},\ 0.117)$
- Scaled softmax of $(0, 2.83, 1.41)$: $(0.045,\ \mathbf{0.768},\ 0.187)$

Already sharper at $d_k = 2$; at $d_k = 512$ scores are $\sim 16\times$ larger and the unscaled softmax would be almost one-hot.

Masked (Causal) Self-Attention: No Peeking at the Future

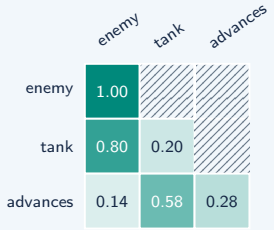
Why a mask?

A model that generates text left to right must not use words it has not written yet. Before the softmax, set every score for a *later* position to $-\infty$; $e^{-\infty} = 0$, so those weights vanish.

CAUSAL MASK

$$A = \text{softmax}\left(\frac{QK^{\mathsf{T}}}{\sqrt{d_k}} + M\right), \quad M_{ij} = \begin{cases} 0 & j \le i \\ -\infty & j > i \end{cases}$$

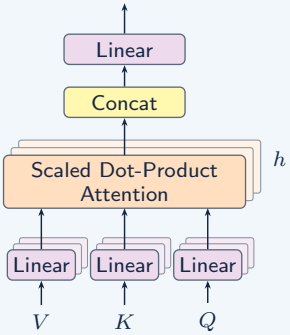
Used in the decoder's first sub-layer in the paper (and in every GPT layer). Encoders use no mask: they read the whole sentence both ways.



our example with a causal mask (hatched = blocked future)

"enemy" can only see itself (1.00); "tank" re-normalises over enemy and tank (0.80, 0.20); the last word is unchanged.

Multi-Head Attention (Vaswani et al., Figure 2, right)



redrawn after Vaswani et al. (2017)

MULTI-HEAD ATTENTION

$$\text{MultiHead}(Q,K,V) = \text{Concat}(\text{head}_1,\ldots,\text{head}_h)\,W^O$$
$$\text{head}_i = \text{Attention}(QW_i^Q,\ KW_i^K,\ VW_i^V)$$

Reading the diagram

1. **Linear** (stacked h times): each head has its own W_i^Q, W_i^K, W_i^V — its own "question"
2. **Scaled dot-product attention**, h copies running in parallel
3. **Concat**: join the h outputs side by side
4. **Linear** W^O: mix them back to d_{model} numbers per word

Sizes in the paper

$h = 8$ heads, $d_k = d_v = d_{\text{model}}/h = 64$; concat $8 \times 64 = 512$; W^O is 512×512. Total cost $\approx$ one full-width head.

Like a staff cell: intelligence, logistics and fires officers read the same report, each extracting different links.

Head 2 weights (head 1 = the worked example)

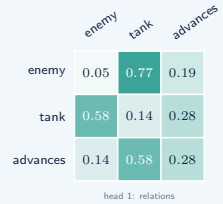
$$W_2^Q = W_2^K = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{pmatrix} \qquad W_2^V = \begin{pmatrix} 0 & 1 \\ 1 & 0 \\ 0 & 1 \\ 1 & 0 \end{pmatrix}$$

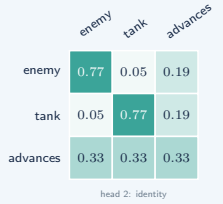
$$Q_2 = K_2 = \begin{pmatrix} 2 & 0 \\ 0 & 2 \\ 1 & 1 \end{pmatrix} \qquad V_2 = \begin{pmatrix} 0 & 2 \\ 2 & 0 \\ 1 & 1 \end{pmatrix}$$

$$S_2 = Q_2 K_2^{\mathsf{T}} = \begin{pmatrix} 4 & 0 & 2 \\ 0 & 4 & 2 \\ 2 & 2 & 2 \end{pmatrix}$$

$$Z_2 = A_2 V_2 = \begin{pmatrix} 0.2775 & 1.7225 \\ 1.7225 & 0.2775 \\ 1.0000 & 1.0000 \end{pmatrix}$$

Row "enemy": scores $(4, 0, 2)/\sqrt{2} \to e^{(\cdot)} = (16.92,\ 1,\ 4.11) \to$ (**0.768**, **0.045**, **0.187**)





Head 1 links words to *each other* (enemy→tank, advances→tank). **Head 2**, with $W_2^Q = W_2^K$, matches each word with *itself* — it preserves each word's own meaning — and "advances" spreads evenly.

Different weights ⇒ different attention patterns from the *same* input. Training lets each head specialise.

Running Example: Concatenate the Heads and Project with W^O

Concat: place Z_1 and Z_2 side by side

$$\text{Concat}(Z_1, Z_2) = \left(\begin{array}{cc|cc} 0.2775 & 1.7679 & 0.2775 & 1.7225 \\ 1.4359 & 1.1400 & 1.7225 & 0.2775 \\ 0.5641 & 1.5760 & 1.0000 & 1.0000 \end{array}\right)$$

Z_1 (head 1) | Z_2 (head 2)

Project: multiply by W^O (4×4)

$\text{MultiHead}(X) = \text{Concat}(Z_1, Z_2)\, W^O$. For hand arithmetic we take $W^O = I$ (identity), so the output equals the concatenation; a trained W^O would mix information across the two heads.

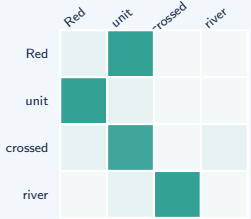
Shape bookkeeping

- each head: 3×2 $(d_v = 2)$
- concat of $h = 2$ heads: 3×4
- after W^O: 3×4 = same shape as X $(d_{\text{model}} = 4)$

Why keep the shape? So the output can be *added* back to the input (the residual connection in Figure 1's "Add & Norm") and the next layer can be stacked on top. W^O maps the concatenation back to d_{model}; choosing $d_v = d_{\text{model}}/h$ makes W^O square and keeps the cost equal to one full-width head.

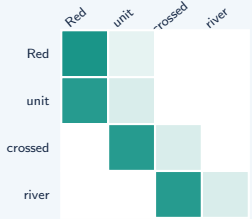
Paper: 8 heads $\times$ 64 $=$ 512 $= d_{\text{model}}$ — exactly our $2 \times 2 = 4$.

What Different Heads Learn in Real Models (Illustrative)



Head A: who did what

Verb "crossed" finds its subject "unit"; object "river" finds its verb.



Head B: previous word

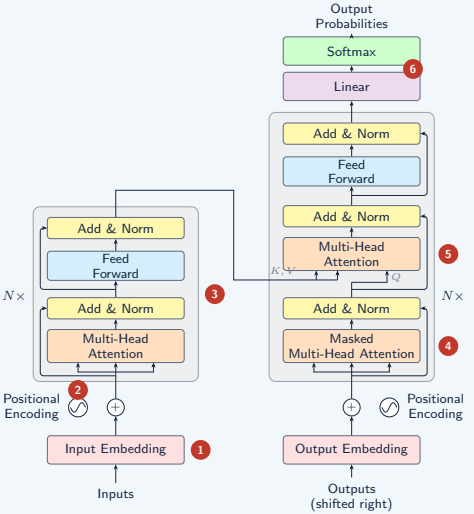
Each word attends to the one before it — a local, positional pattern made possible by the positional encoding.

What research shows

The paper's appendix shows heads that follow long-distance dependencies and resolve pronouns. Later probing studies found heads tracking syntax and copying repeated phrases; many heads are redundant and can be pruned.

Caution: attention maps are a tempting but *partial* explanation. High weight does not prove a word caused the output — treat heat-maps as hypotheses, not evidence.

The Transformer of Attention Is All You Need (Figure 1)



redrawn after Vaswani, Shazeer, Parmar, Uszkoreit, Jones, Gomez, Kaiser & Polosukhin (2017)

Left tower: the encoder (reads the source sentence)

1 input embedding 2 positional encoding 3 encoder layer: multi-head self-attention + feed-forward, each wrapped in Add & Norm; stacked $N = 6$ times

Right tower: the decoder (writes the target sentence)

4 masked self-attention over the words written so far 5 encoder–decoder attention: queries from the decoder, keys and values from the encoder then feed-forward; stacked $N = 6$ times

Top: the output

6 Linear layer + softmax give a probability for every word in the vocabulary.

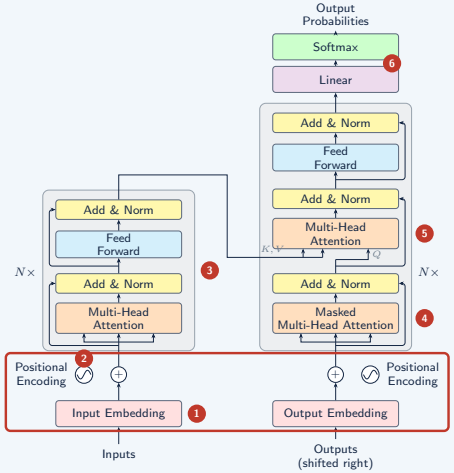
Read it **bottom to top**. Grey boxes are the repeated layers. Every block outputs vectors of the same size, $d_{\text{model}} = 512$.

Following One Sentence Through Figure 1

Task from the paper: English → German translation. "enemy tank advances" → "Feindlicher Panzer rückt vor". Base model, $n = 3$ source tokens, $m = 5$ decoder inputs (words simplified to one token each).

	Block	What happens	Shape
1	Inputs → input embedding	3 token IDs look up 3 rows of the embedding table, scaled by $\sqrt{512}$	3×512
2	+ positional encoding	add sinusoidal codes for positions 0, 1, 2	3×512
3	encoder layer $\times 6$	8-head self-attention (scores 3×3 per head), Add & Norm, FFN ($512 \to 2048 \to 512$), Add & Norm	3×512 ("memory")
4	outputs (shifted right)	$\langle s \rangle$ Feindlicher Panzer rückt vor: embed + position, masked self-attention (scores 5×5, lower triangle)	5×512
5	encoder–decoder attention	queries from the 5 decoder positions, keys & values from the 3-word memory (scores 5×3)	5×512
6	Linear + softmax	scores for each of ≈37 000 vocabulary tokens, turned into probabilities	$5 \times 37\,000$

Training target for the 5 positions: "Feindlicher Panzer rückt vor $\langle /s \rangle$" — each position predicts the *next* German word. All 5 predictions are made in one parallel pass.



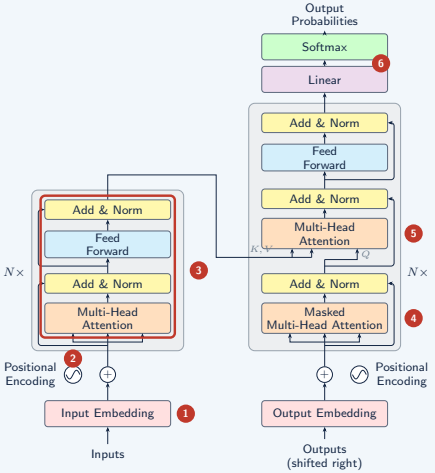
1 Embeddings

- Token IDs $\rightarrow$ learned vectors of size $d_{\text{model}} = 512$
- The paper **shares one weight matrix** between the input embedding, the output embedding and the final pre-softmax Linear layer
- Embeddings are multiplied by $\sqrt{d_{\text{model}}} = 22.6$ so the position codes do not drown them

2 Positional encoding $\oplus$

- The wave symbol in the figure $=$ the sinusoidal PE; the $\oplus =$ element-wise **addition**
- Same formula on both towers; the decoder's positions count the German words
- Dropout ($P_{\text{drop}} = 0.1$) is applied to the sums

Running example: $X = E + P$ (computed earlier) is this stage.



Sub-layer 1: multi-head self-attention

The three arrows entering the block are Q, K and V — all copies of the same input: every source word attends to every source word (no mask).

Sub-layer 2: position-wise feed-forward network

$\text{FFN}(x) = \max(0,\ xW_1 + b_1)\,W_2 + b_2,\ \ 512 \to 2048 \to 512$
The same two-layer ReLU network applied to each word separately — "attention mixes words, the FFN thinks about each word".

Add & Norm around each sub-layer

$\text{output} = \text{LayerNorm}\big(x + \text{Sublayer}(x)\big)$
The arrow bypassing each block is the **residual connection** (the "Add"); LayerNorm rescales each word vector to mean 0, variance 1 (the "Norm").

"$N\times$": six identical layers with *separate* weights; the output of layer 6 is the memory passed to the decoder.

Add: input X plus multi-head output

$$H = X + \text{MultiHead}(X) = \begin{pmatrix} 1.2775 & 1.7679 & 1.2775 & 1.7225 \\ 1.4359 & 2.1400 & 1.7225 & 1.2775 \\ 1.5641 & 1.5760 & 1.0000 & 2.0000 \end{pmatrix}$$

e.g. enemy: $(1, 0, 1, 0) + (0.2775, 1.7679, 0.2775, 1.7225)$

Norm: each row to mean 0, variance 1

Row enemy: mean $\mu = 1.5113$, variance $\sigma^2 = 0.0550$, $\sigma = 0.2344$

$$\text{LN}(h) = \frac{h - \mu}{\sigma} = (\textbf{-0.998},\ \textbf{1.094},\ \textbf{-0.998},\ \textbf{0.901})$$

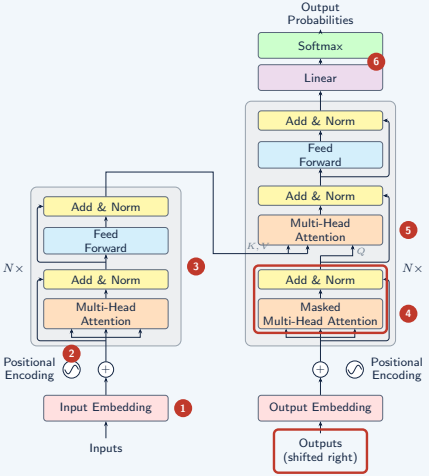
$$\text{LN}(H) = \begin{pmatrix} -0.998 & 1.094 & -0.998 & 0.901 \\ -0.635 & 1.513 & 0.240 & -1.118 \\ 0.082 & 0.115 & -1.506 & 1.309 \end{pmatrix}$$

Residual (Add): the original word vector passes straight through; attention only adds a *correction*. Gradients flow through the "+" unchanged, so 6 (or 96) layers still train.

Layer normalisation (Norm)

Keeps every word vector in a stable range whatever happened upstream. A learned scale γ and shift β follow (here $\gamma = 1$, $\beta = 0$).

Next, each row goes through the FFN, then a second Add & Norm — completing one encoder layer. The result has the same 3×4 shape, ready for layer 2.



Teacher forcing during training

position	1	2	3	4	5
decoder input	$\langle s \rangle$	Feindlicher	Panzer	rückt	vor
word to predict	Feindlicher	Panzer	rückt	vor	$\langle / s \rangle$

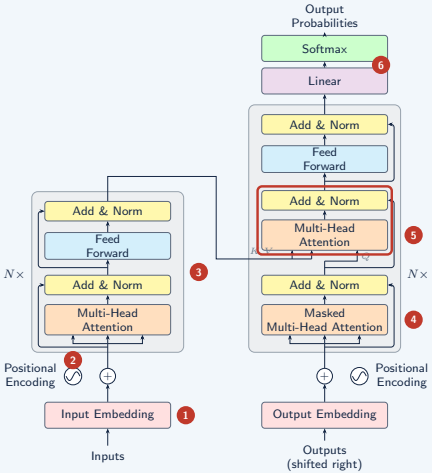
Why shifted right?

The correct German sentence is fed in, moved one place to the right behind a start symbol $\langle s \rangle$. Position t must predict word t while seeing only words $1 \ldots t-1$.

Why masked?

The causal mask (the $-\infty$ trick) stops position 3 (“Panzer”) from peeking at “rückt” and “vor”. The paper: masking plus the one-position offset ensure “predictions for position i can depend only on the known outputs at positions less than i”.

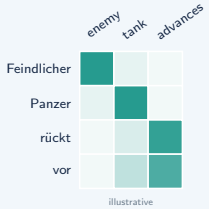
At inference there is no target: the decoder starts with $\langle s \rangle$ and feeds back each word it produces.



Where Q, K, V come from here

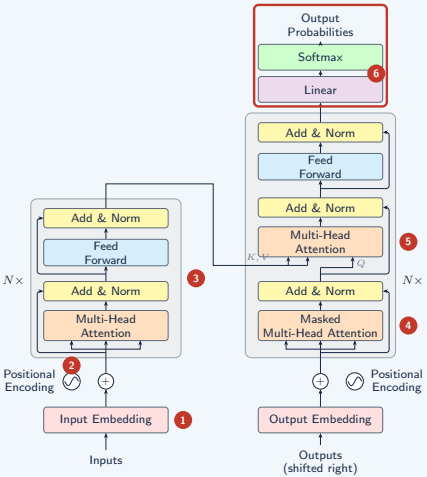
- **Queries**: from the decoder ("which source words do I need to produce my next German word?")
- **Keys and values**: from the encoder's final output (the two arrows entering from the left tower)

This is the only place the two towers connect.



The attention map acts as a learned **word alignment**. Note that both halves of the German separable verb "rückt ... vor" look back to "advances".

Scores are 5×3 (decoder positions $\times$ source words) — no mask is needed: the whole source sentence is known.

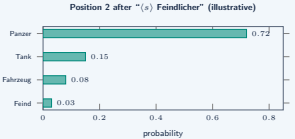


Linear

Multiplies each final decoder vector (512 numbers) by a $512 \times V$ matrix, giving one score (logit) per vocabulary token — about 37 000 for the paper's English–German model. This matrix is **shared** with the embedding table.

Softmax

Turns the scores into a probability distribution. Training minimises cross-entropy $-\ln P(\text{correct next word})$ at every position (with label smoothing $\epsilon_{ls} = 0.1$).



At inference the paper used **beam search** (beam size 4) rather than always taking the single most likely word.

Three Uses of Attention in One Model

	Encoder self-attention 3	Decoder masked self-attention 4	Encoder–decoder attention 5
Queries from	source words	target words so far	target words so far
Keys & values from	source words	target words so far	encoder output (source)
Mask	none	causal ($-\infty$ above diagonal)	none
Score matrix (example)	3×3	5×5, lower triangle	5×3
Question it answers	"how do the English words relate?"	"what have I written so far?"	"which English words do I translate next?"

The *same* scaled dot-product attention formula is used in all three places; only the sources of Q, K, V and the mask differ.

The Paper's Settings and Results

	Base	Big
layers N (each tower)	6	6
d_{model}	512	1024
d_{ff} (FFN hidden)	2048	4096
heads h	8	16
$d_k = d_v$	64	64
dropout P_{drop}	0.1	0.3
parameters	65 M	213 M
training (8 NVIDIA P100 GPUs)	12 h	3.5 days
BLEU, WMT 2014 English–German	27.3	**28.4**
BLEU, WMT 2014 English–French	38.1	**41.8**

Training recipe

- Adam optimiser, $\beta_1 = 0.9$, $\beta_2 = 0.98$
- learning rate warms up for 4 000 steps, then decays: $\text{lr} = d_{\text{model}}^{-0.5} \min(\text{step}^{-0.5},\ \text{step} \cdot 4000^{-1.5})$
- dropout and label smoothing for regularisation
- English–German: ≈4.5 M sentence pairs, ≈37 000-token shared vocabulary

The big model beat the previous best English–German results, including ensembles, by more than 2 BLEU — at a fraction of their training cost.

English–French big model: dropout 0.1. Figures from the paper's Tables 2 and 3.

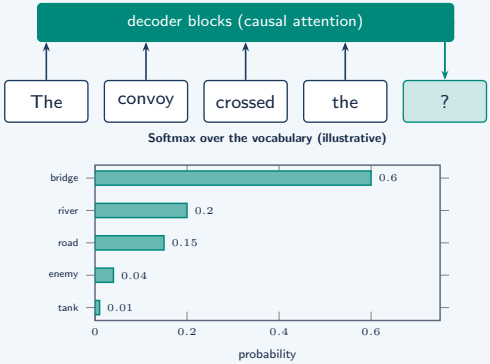
Three Families of Transformer Models

	Encoder-only	Decoder-only	Encoder–decoder
Which half of Fig. 1	left tower	right tower without 5	both towers (the original)
Examples	BERT, RoBERTa	GPT series, Claude, Llama	original Transformer, T5, BART
Attention	bidirectional, no mask	causal mask	encoder bidirectional; decoder causal + cross
Trained to	fill in masked words	predict the next token	map input text to output text
Defence example	tag units, places and equipment in intercepts	draft a SITREP from field notes	translate captured documents

Same building block throughout; the families differ only in the mask, the wiring and the training objective.

Scale

BERT-base ≈110 M parameters (2018); GPT-2 1.5 B (2019); GPT-3 175 B (2020).



The loop

1. Run the prompt through all blocks
2. The last position's vector $\times$ an output matrix gives a score (logit) for every token in the vocabulary
3. Softmax $\rightarrow$ probabilities; pick or sample a token ("bridge")
4. Append it and repeat

Temperature τ: softmax of logits/τ. Low τ $\rightarrow$ predictable; high τ $\rightarrow$ varied. Logits here: $(3.2, 2.1, 1.8, 0.4, -0.5)$.

Pre-training: predict the next token

For every position t in billions of sentences, the loss is the cross-entropy of the true next token:

$$L = -\frac{1}{T}\sum_{t=1}^{T} \ln P_\theta(w_{t+1} \mid w_1, \ldots, w_t)$$

In the example, if the true word is "bridge": $L = -\ln 0.60 = 0.51$.

The causal mask lets one forward pass produce a prediction — and a loss term — at *every* position simultaneously. That parallelism is why transformers train so efficiently.

Then backpropagation, as before

The gradient flows back through softmax, W^O, every head's W^Q, W^K, W^V, the FFNs and the embedding table — the same chain rule as Lecture 6 (backpropagation by hand), just with more layers.

After pre-training

- **Fine-tuning** on a smaller, task-specific dataset
- **Instruction tuning** and **human-feedback** training to follow requests helpfully and safely
- Or no training at all: give examples in the prompt (*in-context learning*)

Text and intelligence

Summarising reports, extracting entities (units, locations, equipment) from intercepts, translating captured documents, searching archives by meaning rather than keyword.

Imagery (Vision Transformer)

Cut a SAR or EO image into 16×16 patches; each patch is a "token" with its own positional encoding. Attention links distant parts of a scene — e.g. vehicles and the road network they use.

Sensor fusion and tracks

Each radar track or sensor report becomes a token; attention learns which tracks relate (formation flying, escort–target pairs).

Wargaming and decision support

Transformer policies over sets of units (any number of tokens), analysis of game logs to find decisive moments, natural-language interfaces to simulations.

Limits to brief commanders on

- **Hallucination**: fluent, confident, wrong statements
- **Prompt injection**: hostile text inside a document can redirect the model
- **Distribution shift**: trained on open text, deployed on jargon and new theatres
- **Cost**: attention is $O(n^2)$ in sequence length

Common Misconceptions

"Attention = understanding." Attention is a weighted average learned to reduce prediction error. Whether that amounts to understanding is a separate question.

"Q, K, V are three different inputs." In self-attention they are three *projections of the same* input X.

"Positional encoding is concatenated." It is *added*, element by element, to the embedding.

"Embedding dimensions have names." Only in our toy; real dimensions are unlabelled and meaning is spread across them.

"The model reads left to right." Only decoders are causal. Encoders see the whole sequence at once.

"Softmax picks the best word." It produces a full distribution; the decoding strategy (greedy, beam search, sampling) picks.

"Bigger context is free." Doubling the sequence length quadruples the attention score matrix: $n \times n$.

Self-check: if you can reproduce the worked example and label every block of Figure 1, you understand the core of every transformer.

Summary and Quick Check

INPUT

$$h_0 \;=\; \sqrt{d_{\text{model}}}\,\text{Embed}(\text{tokens}) \;+\; PE$$

POSITIONAL ENCODING

$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d}),\;\; PE_{(pos,2i+1)} = \cos(\cdot)$$

ATTENTION

$$\text{softmax}\big(QK^{\mathsf{T}}/\sqrt{d_k}\,[+M]\big)\,V,\;\; Q \;=\; XW^{Q},\; K \;=\; XW^{K},\; V \;=\; XW^{V}$$

MULTI-HEAD

$$\text{Concat}(\text{head}_1,\ldots,\text{head}_h)\,W^{O}$$

EACH SUB-LAYER

$$\text{LayerNorm}(x \;+\; \text{Sublayer}(x))$$

Quick check

1. Using the toy table, compute *lion* $-$ *man* $+$ *woman* (lion $= (0.85, 0, 0.90, 0.30)$). Which word is nearest?
2. Compute p_3 for $d_{\text{model}} = 4$.
3. In head 1, which word does "tank" attend to most, and with what weight?
4. Why must the multi-head output have the same width as X?
5. In Figure 1, where do the keys and values of block 5 come from, and why is no mask needed there?

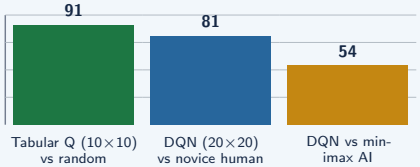
From passive learning to active decision-making

- RL vs supervised / unsupervised
- MDP: S, A, $P(s'|s,a)$, $R(s,a,s')$, γ
- Bellman optimality: $V^*(s)$, $Q^*(s,a)$ derived from scratch
- Q-learning: TD update, ε-greedy, convergence
- DQN: a neural network — trained by backpropagation (L6) — replaces the Q-table; experience replay; target network

Assignment 9: sinusoidal positional encoding + scaled dot-product attention by hand + multi-head shapes and parameter counts + masking and cost + BERT vs GPT for a defence task; bonus: Add & Norm by hand.

Preview: wargame tactical agent

20×20 hex JANUS map; state space $\sim 10^{28}$ (no Q-table can hold it); actions {move N/S/E/W, attack, defend, resupply}; reward $+10$ objective, $+5$ kill, -8 unit lost.



DQN: 3 layers × 512 units; 100K-transition replay buffer; target update every 1,000 steps; 100K episodes, 48 GPU-hours.